



ASYNCHRONOUS PROGRAMMING

By Nada Yousry

THE PROBLEM: WAITING

Some operations are slow like :

- Network Request
- Database Query
- File I/O
- Timers

.....What happens while we wait?



The application
waits



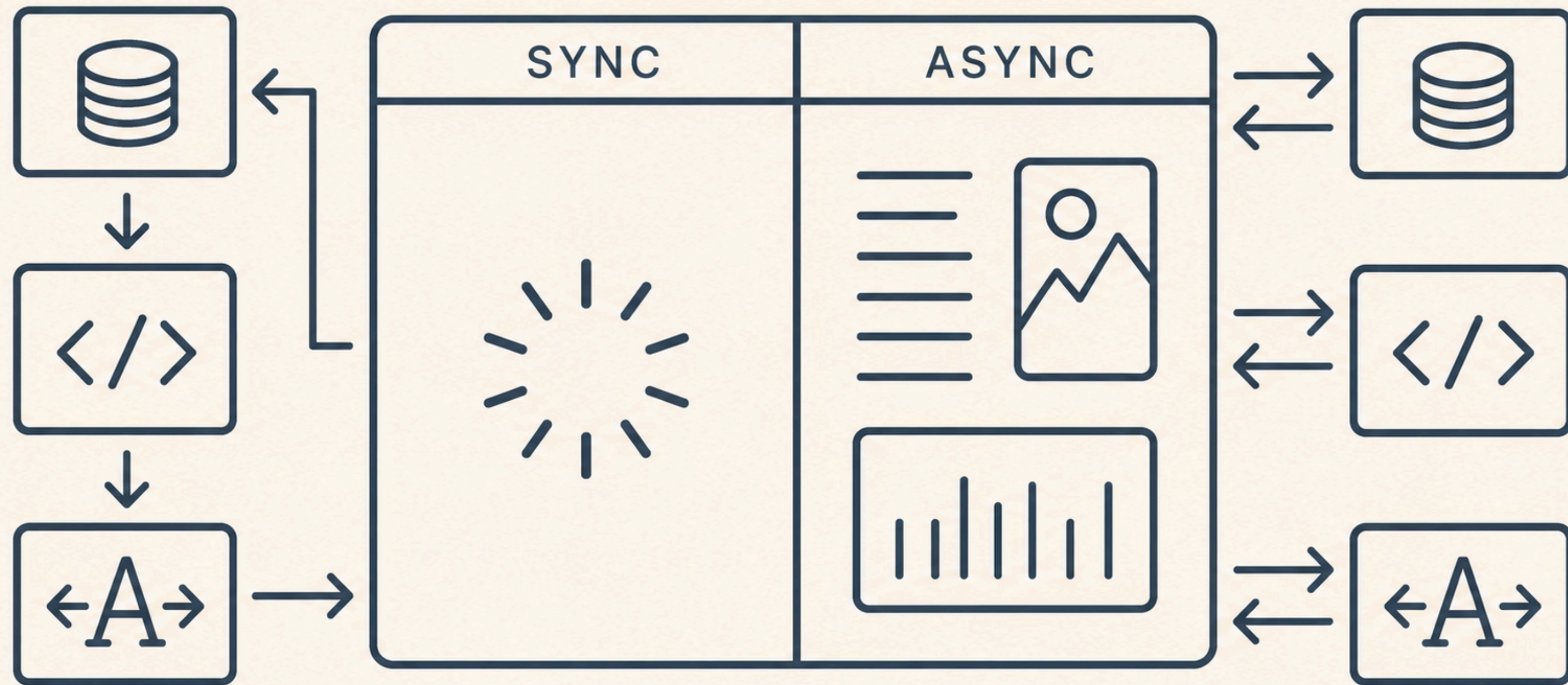
The UI may freeze



The user thinks the
app is stuck



HOW DOES THE APPLICATION HANDLE WAITING



HOW DO WE HANDLE ASYNCHRONOUS OPERATIONS?

When an operation takes time, we need a way to know when it finishes and what result it produces.

- Callbacks

A function is executed when the operation finishes.

- Promises / Futures

Represent a result that will be available in the future.

- Async / Await

Provides a cleaner and more readable way to work with asynchronous operations.

CALLBACKS “I’ll let you know when I’m done.”

A function that runs when the operation finishes

Each operation tells us what to do when it finishes.

GET POSTS



WAIT FOR RESPONSE



LOAD COMMENTS



WAIT FOR RESPONSE



SHOW COMMENTS

LONG OPERATION



TAKES TIME



CALLBACK FUNCTION



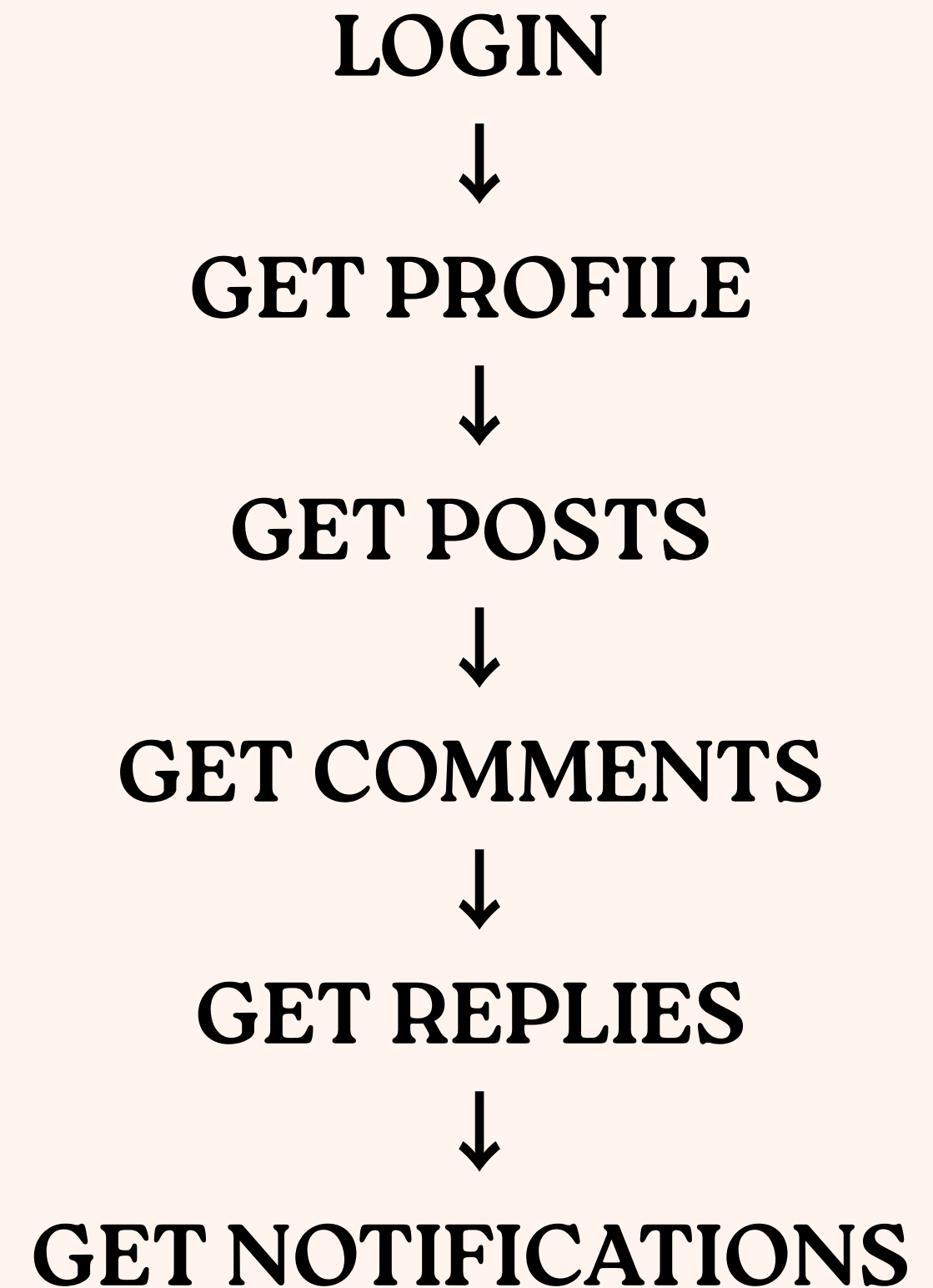
OPERATION FINISHED



HANDLE THE RESULT

CALLBACK HELL

Hard to read • Hard to maintain • Hard to debug



MORE CALLBACKS → MORE NESTING → CALLBACK HELL

FUTURE / PROMISE

A Promise is an object that represents the eventual result of an asynchronous operation

We use `.then()` to define what happens once the result is ready

It lets us hold the result as a "thing" we can act on whenever we choose — not just at the moment of the call



```
getComments(post)  
  .then(comments => print(comments))
```

THE PROBLEM WITH .THEN()

- Chaining several async operations with .then() still leads to visual complexity
- Each step needs its own small function passed into .then()
- The code doesn't read like normal, step-by-step (synchronous) code
- This makes it harder to follow the logic, even though the nesting problem is solved



```
getPost()  
  .then(post => getComments(post))  
  .then(comments =>  
    getAuthor(comments))  
  .then(author => print(author))
```

ASYNC / AWAIT IN ACTION

`async` → Marks the function as asynchronous

`await` → Pauses this function until the result is ready, without blocking the rest of the program

Lets us write asynchronous code that looks and reads like normal, synchronous code
Removes the need for chained `.then()` calls



```
async function getData() {  
    post = await getPost()  
    comments = await getComments(post)  
    author = await getAuthor(comments)  
    print(author)  
}
```

WHY IS IT IMPORTANT?

Better User Experience

The UI stays responsive while waiting for slow operations.

Better Performance

The application can use waiting time to perform other tasks.

Handle Multiple Operations

Applications can deal with multiple network, database, or file operations without blocking unnecessarily.

Common in Modern Applications

APIs • Databases • File I/O • Networking





THANK YOU